

Assignment No. 6

Q1)

Call : $f(3)$;

O/p:-
3 2 1 2 3 4

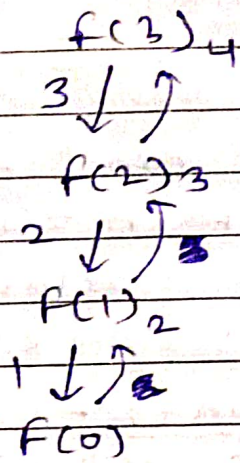
```
void f f (int n) {
    if (n == 0) {
        return;
```

$f(0)$
$f(1)$
$f(2)$
$f(3)$

```
    sop (n + " ");
    f(n-1);
    sop ((n+1) + " ");
```

Recursive Tree

call stack



3

```
void fun (int n) {
    if (n == 0) {
        return;
```

```
    sop (n + " ");
    f(n-1);
    sop ((n+1) + " ");
```

3

```
void fun (int n) {
    if (n == 0) {
        return;
```

```
    sop (n + " ");
    f(n-1);
    sop ((n+1) + " ");
```

3

```
void fun (int n) {
    if (n == 0) {
        return;
```

```
    sop (n + " ");
    f(n-1);
    sop ((n+1) + " ");
```

3

02)

Call: ~~fun(2);~~ fun(2);

void fun(int n) {

if (n == 0) {

return;

sop("x");

fun(n-1);

sop("y");

fun(n-1);

}

void fun(int n) {

if (n == 0) {

return;

sop("x");

fun(n-1);

sop("y");

fun(n-1);

}

void fun(int n) {

if (n == 0) {

return;

sop("x");

fun(n-1);

sop("y");

fun(n-1);

}

void fun(int n) {

if (n == 0) {

return;

}

01010

xx 44xx

void fun(int n)

if (n == 0)

return;

sop("x");

fun(n-1);

sop("y");

fun(n-1);

}

void fun(int n)

if (n == 0)

return;

sop("x");

fun(n-1);

sop("y");

fun(n-1);

}

void fun(int n)

if (n == 0)

return;

sop("x");

fun(n-1);

sop("y");

fun(n-1);

}

fun(2)

fun(1)

fun(0)

fun(1)

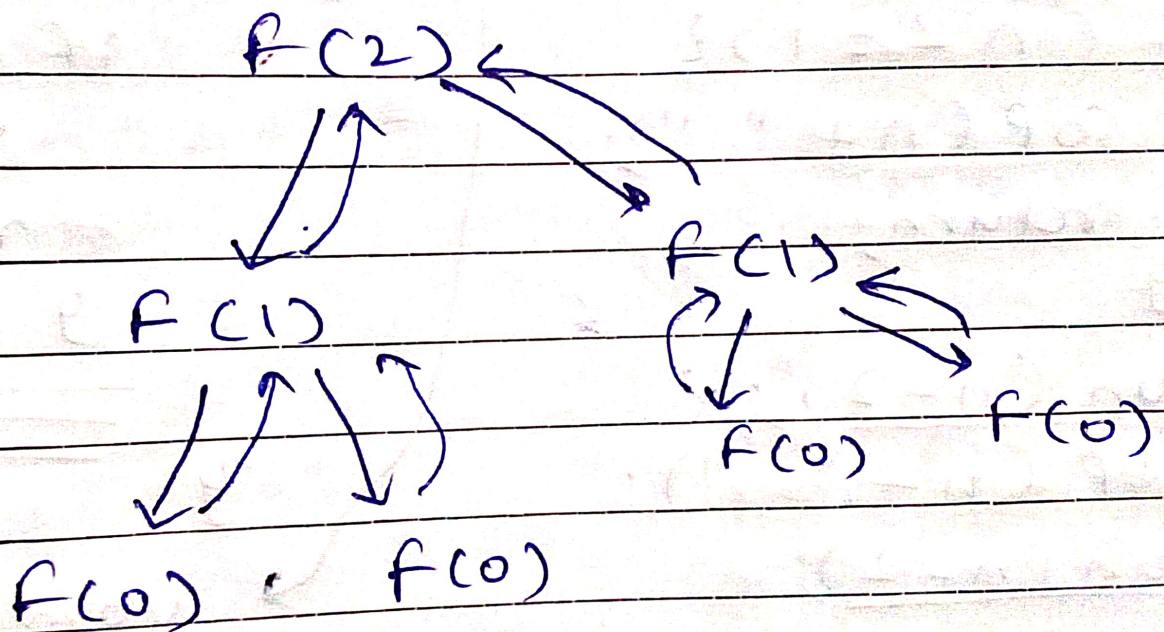
fun(0)

fun(1)

fun(2)

call stack

Recursive tree



Q3)

Call: fun(4);

0/p1-

0 2 0 4 0 2 0

```
void fun(int n) {
    if (n <= 1) {
        SOP(n + " ");
        return;
    }
    fun(n-2);
    SOP(n + " ");
    fun(n-2);
}
```

```
void fun(int n) {
    if (n <= 1) {
        SOP(n + " ");
        return;
    }
    fun(n-2);
    SOP(n + " ");
    fun(n-2);
}
```

fun(4)
fun(2)
fun(0)
fun(2)
fun(0)
fun(4)

Call stack

```
void fun(int n) {
    if (n <= 1) {
        SOP(n + " ");
        return;
    }
    fun(n-2);
    SOP(n + " ");
    fun(n-2);
}
```

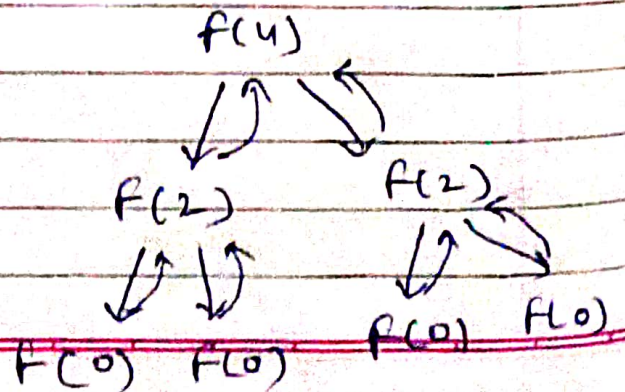
```
void fun(int n) {
    if (n <= 1) {
        SOP(n + " ");
        return;
    }
    fun(n-2);
    SOP(n + " ");
    fun(n-2);
}
```

```
void fun(int n) {
    if (n <= 1) {
        SOP(n + " ");
        return;
    }
    fun(n-2);
    SOP(n + " ");
    fun(n-2);
}
```

```
void fun(int n) {
    if (n <= 1) {
        SOP(n + " ");
        return;
    }
    fun(n-2);
    SOP(n + " ");
    fun(n-2);
}
```

```
void fun(int n) {
    if (n <= 1) {
        SOP(n + " ");
        return;
    }
    fun(n-2);
    SOP(n + " ");
    fun(n-2);
}
```

Recursive tree



Q4) Call: fun(2);

o/p: 1 4 1

```
void fun(int n) {
    if (n == 0)
        return;
    fun(n-1);
    SOP(n*n + " ");
    fun(n-1);
}
```

```
void fun(int n) {
    if (n == 0)
        return;
    fun(n-1);
    SOP(n*n + " ");
    fun(n-1);
}
```

fun(2)
fun(1)
fun(0)
fun(1)
fun(2)
fun(1)
fun(0)

Call stack

```
void fun(int n) {
    if (n == 0)
        return;
    fun(n-1);
    SOP(n*n + " ");
    fun(n-1);
}
```

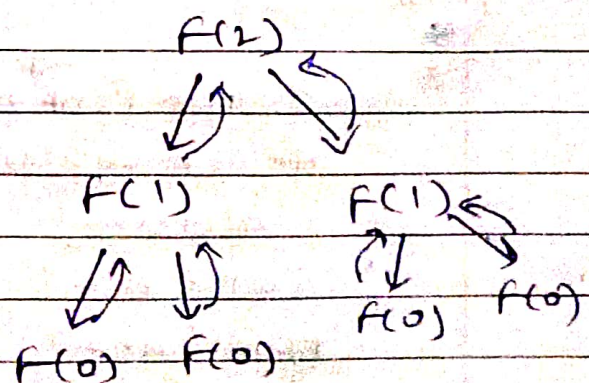
```
void fun(int n) {
    if (n == 0)
        return;
    fun(n-1);
    SOP(n*n + " ");
    fun(n-1);
}
```

```
void fun(int n) {
    if (n == 0)
        return;
    fun(n-1);
    SOP(n*n + " ");
    fun(n-1);
}
```

```
void fun(int n) {
    if (n == 0)
        return;
    fun(n-1);
    SOP(n*n + " ");
    fun(n-1);
}
```

Recursive tree

```
void fun(int n) {
    if (n == 0)
        return;
    fun(n-1);
    SOP(n*n + " ");
    fun(n-1);
}
```



Q5] Call: fun(5);

o/p r

5 3 1 1 3 5

```
void fun(int n) {
```

```
    if (n <= 0)
```

```
        return;
```

```
    sop(n + " ");
```

```
    fun(n - 2);
```

```
    sop(n + " ");
```

4

```
void fun(int n) {
```

```
    if (n <= 0)
```

```
        return;
```

```
    sop(n + " ");
```

```
    fun(n - 2);
```

```
    sop(n + " ");
```

3

```
void fun(int n) {
```

```
    if (n <= 0)
```

```
        return;
```

```
    sop(n + " ");
```

```
    fun(n - 2);
```

```
    sop(n + " ");
```

2

```
void fun(int n) {
```

```
    if (n <= 0)
```

```
        return;
```

```
    sop(n + " ");
```

```
    fun(n - 2);
```

```
    sop(n + " ");
```

1

fun(5)

fun(3)

fun(1)

fun(-1)

calls stack

Recursive tree

F(5)

F(3)

F(1)

F(-1)

Q6) Call: fun(8)

o/p:

8 4 2 1 2 4 8

```
void fun(int n) {  
    if (n <= 1) {  
        sop(n + " ");  
        return;  
    }
```

```
    sop(n + " ");  
    fun(n/2);  
    sop(n + " ");  
}
```

```
void fun fun(int n) {  
    if (n <= 1) {  
        sop(n + " ");  
        return;  
    }
```

```
    sop(n + " ");  
    fun(n/2);  
    sop(n + " ");  
}
```

```
void fun(int n) {  
    if (n <= 1) {  
        sop(n + " ");  
        return;  
    }
```

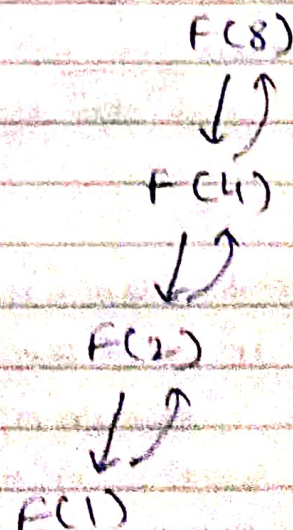
```
    sop(n + " ");  
    fun(n/2);  
    sop(n + " ");  
}
```

fun(8)
fun(4)
fun(2)
fun(1)

call stack

```
void fun(int n) {  
    if (n <= 1) {  
        sop(n + " ");  
        return;  
    }  
    sop(n + " ");  
    fun(n/2);  
    sop(n + " ");  
}
```

Recursive tree



Q-7) Call: fun(3);

O/p: A3 A2 A1 B2 B4 B6

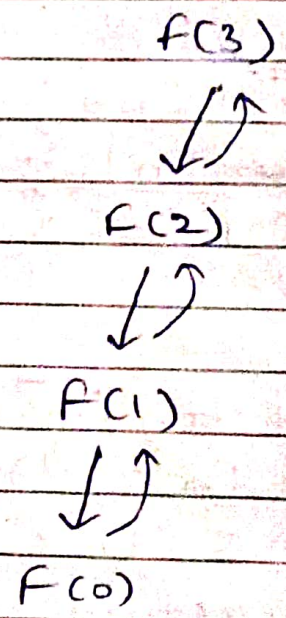
```
void fun(int n) {
    if (n <= 0)
        return;
    SOP("A" + n + " ");
    fun(n-1);
    SOP("B" + (n*2) + " ");
}
```

fun(0)
fun(1)
fun(2)
fun(3)

call stack

```
void fun(int n) {
    if (n <= 0)
        return;
    SOP("A" + n + " ");
    fun(n-1);
    SOP("B" + (n*2) + " ");
}
```

Recursive tree



```
void fun(int n) {
    if (n <= 0)
        return;
    SOP("A" + n + " ");
    fun(n-1);
    SOP("B" + (n*2) + " ");
}
```

```
void fun(int n) {
    if (n <= 0)
        return;
    SOP("A" + n + " ");
    fun(n-1);
    SOP("B" + n + " ");
}
```

}

Q8) Call: fun(3);

O/p:-

1 2 0 3 1

```

void fun(int n) {
    if (n <= 1) {
        sop(n + " ");
        return;
    }
    fun(n-1);
    sop(n + " ");
    fun(n-2);
}
    
```

```

void fun(int n) {
    if (n <= 1) {
        sop(n + " ");
        return;
    }
}
    
```

1	2	0	3	1
1	2	0	3	1
1	2	0	3	1
1	2	0	3	1
1	2	0	3	1

Call stack

```

void fun(int n) {
    if (n <= 1) {
        sop(n + " ");
        return;
    }
    fun(n-1);
    sop(n + " ");
    fun(n-2);
}
    
```

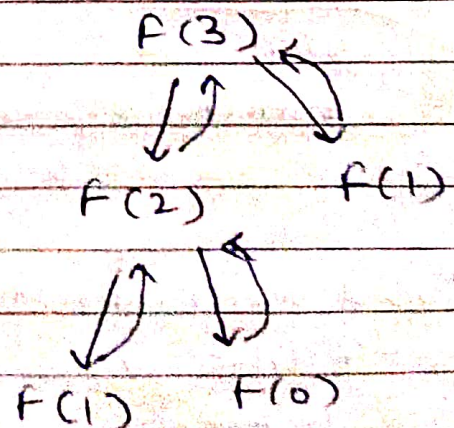
```

void fun(int n) {
    if (n <= 1) {
        sop(n + " ");
        return;
    }
}
    
```

Recursive tree

```

void fun(int n) {
    if (n <= 1) {
        sop(n + " ");
        return;
    }
    fun(n-1);
    sop(n + " ");
    fun(n-2);
}
    
```



Q9]

Call: fun(4);

O/p:-

1 2 3 4 4 2 1

void fun(int n) {
 if (n <= 0)
 return;
 fun(n-1);
 SOP(n+" ");
 fun(n-2);
}

void fun(int n) {
 if (n <= 0)
 return;
 fun(n-1);
 SOP(n+" ");
 fun(n-2);
}

void fun(int n) {
 if (n <= 0)
 return;
 fun(n-1);
 SOP(n+" ");
 fun(n-2);
}

void fun(int n) {
 if (n <= 0)
 return;
 fun(n-1);
 SOP(n+" ");
 fun(n-2);
}

void fun(int n) {
 if (n <= 0)
 return;
 fun(n-1);
 SOP(n+" ");
 fun(n-2);
}

void fun(int n) {
 if (n <= 0)
 return;
 fun(n-1);
 SOP(n+" ");
 fun(n-2);
}

void fun(int n) {
 if (n <= 0)
 return;
 fun(n-1);
 SOP(n+" ");
 fun(n-2);
}

void fun(int n) {
 if (n <= 0)
 return;
 fun(n-1);
 SOP(n+" ");
 fun(n-2);
}

fun(4)
 fun(3)
 fun(2)
 fun(1)
 fun(0)
 fun(-1)
 fun(-2)
 fun(-3)
 fun(-4)
 call stack

```
void fun (int n) {
    if (n <= 0)
        return;
    fun (n-1);
    SOP (n + " ");
    fun (n-2);
}
```

3

```
void fun (int n) {
    if (n <= 0)
        return;
    fun (n-1);
    SOP (n + " ");
    fun (n-2);
}
```

3

```
void fun (int n) {
    if (n <= 0)
        return;
    fun (n-1);
    SOP (n + " ");
    fun (n-2);
}
```

3

```
void fun (int n) {
    if (n <= 0)
        return;
    fun (n-1);
    SOP (n + " ");
    fun (n-2);
}
```

3

```
void fun (int n) {
    if (n <= 0)
        return;
}
```

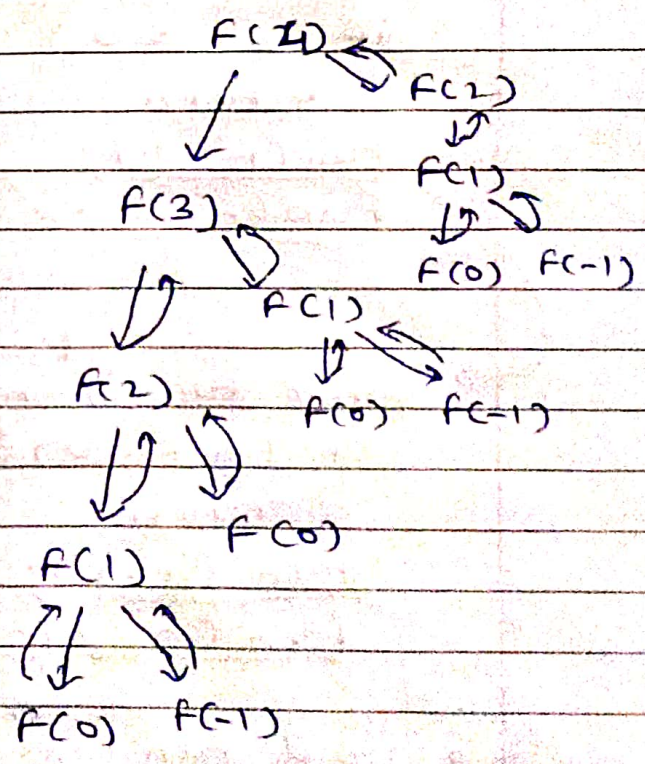
3

```
void fun (int n) {
    if (n <= 0)
        return;
}
```

3

```
void fun (int n) {
    if (n <= 0)
        return;
}
```

3



fun(4)
fun(3)
fun(2)
fun(1)
fun(0)
fun(1)
fun(2)
fun(3)
fun(4)
fun(3)
fun(2)
fun(1)
fun(0)
fun(1)
fun(2)
fun(3)
fun(4)

Q10] Call: fun(4);

4
void fun(int n){
if (n <= 0)
return;
fun(n-1);
fun(n-2);

1
void fun(int n){
if (n <= 0)
return;
fun(n-1);
fun(n-2);

3
void fun(int n){
if (n <= 0)
return;
fun(n-1);
fun(n-2);

0
void fun(int n){
if (n <= 0)
return;
fun(n-1);
fun(n-2);

2
void fun(int n){
if (n <= 0)
return;
fun(n-1);
fun(n-2);

1
void fun(int n){
if (n <= 0)
return;
fun(n-1);
fun(n-2);

1
void fun(int n){
if (n <= 0)
return;
fun(n-1);
fun(n-2);

0
void fun(int n){
if (n <= 0)
return;
fun(n-1);
fun(n-2);

0
void fun(int n){
if (n <= 0)
return;

1
void fun(int n){
if (n <= 0)
return;
fun(n-1);
fun(n-2);

call stack

4

```

2
void fun (int n) {
    if (n <= 0) {
        return;
        fun (n-1);
        fun (n-2);
    }
}

```

```

0
void fun (int n) {
    if (n <= 0)
        return;
    fun (n-1);
    fun (n-2);
}

```

```

1
void fun (int n) {
    if (n <= 0)
        return;
    fun (n-1);
    fun (n-2);
}

```

```

0
void fun (int n) {
    if (n <= 0)
        return;
    fun (n-1);
    fun (n-2);
}

```

```

-1
void fun (int n) {
    if (n <= 0)
        return;
    fun (n-1);
    fun (n-2);
}

```

Recursive tree

